Document Number: P1666R0
Date: 2019-06-09
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22

Summaries of SG16 meetings are maintained at https://github.com/sg16-unicode/sg16-meetings. This paper contains a snapshot of select meeting summaries from that repository.

January 23rd, 2019

Draft agenda:

- Peter Bindels will present his work on a simple 2D graphics library.
- Discuss Steve's latest draft of the SG16 rubric.
- Discuss Tom's latest draft of the char8_t remediation paper.

Attendees:

- Bryce Adelstein Lelblach
- Corentin Jabot
- JeanHeyd Meneide
- Michael Spencer
- Peter Bindels
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Peter Bindels presented his work on a simple 2D graphics library.
  - https://github.com/dascandy/pixel
  - Peter summarrized applicability to SG16:
    - In a graphics API, what is the first thing you want to put on the screen? Text of course!
    - Putting text on the screen requires Unicode support for the library to be generally usable.
    - A text type handling Unicode is therefore needed.

    - Michael stated that how to display text is not part of text processing and therefore not part of SG16 scope, but rather fits into SG13 scope. SG16 won't standardize harfbuzz. The question for SG16 is, how to accept text.
    - Zach suggested the perspective that, give me a sequence of code points and I will render them. The Unicode line break algorithms handle layout pretty well.
    - Peter noted that the bidirectional algorithm is needed as well.
    - Zach added that normalization probably isn't required though.
    - Tom asked if Peter wanted to support italic, bold, or underlined text. A long and contentious debate had been on-going on the Unicode Consortium mailing list regarding whether Unicode should enable encoding stress indicators like these. See the threads at https://unicode.org/pipermail/unicode/2019-January/007313.html and https://unicode.org/pipermail/unicode/2019-January/007434.html.
    - Peter responded, no, plain text is sufficient.
    - Zach stated that stress indicators are handled by fonts and controlled by markup.
    - Tom elaborated, the question was intended to discover if Peter's needs are limited to plain text or whether a higher level of markup is needed. Would there be a desire to standardize some kind of markup?
    - Steve suggested that inline markup could be supported. Or not supported as desired.
    - Zach added that text display is relatively simple once decoding, line breaks, and bidirectional support are enabled; fonts take care of the rest.
    - Steve stated that our immediate goals are low level; provide code point and EGC decoding support.
    - Peter summarized, so we're on the right track working to get new types into the standard.
    - Tom agreed, new types and new algorithms.
    - Steve added, and views.
    - Peter, changing topics, Christopher DiBella has been asking for SG20 how to educate people about Unicode.
- P1253R0: Guidelines for when a WG21 proposal should be reviewed by SG16, the text and Unicode study group
    - Tom asked, do we want to present this to the various WGs or just the WG chairs?
    - Zach stated he was most concerned about EWGI and LEWGI.
    - Corentin suggested presenting to LEWGI, WG chairs, and mentioning it at plenary.
    - Bryce suggested that we could present it at an evening session and could arrange presentation at EWGI and LEWGI at Kona.
    - Tom stated he would follow up with WG chairs [Editor's note: Tom did later reach out to the EWG, LEWG, CWG, LWG, EWGI, and LEWGI chairs to ensure that
        - a. the chairs agree with the guidelines, and
        - b. are willing to direct proposal authors to SG16 when discussion touches on the topics discussed in the paper.
    - Several chairs have indicated their agreement so far. Tom also requested presenting the paper to EWGI and LEWGI in Kona.]
    - Peter mentioned that there may be examples where a filename may be mutated on open (for normalization purposes) such that attempts to reopen the file by the mutated name fail. We should 1) verify that this can happen and, if so, 2) update the paper to mention it.
    - Tom advised that, while we're all participating in WG discussions, that we continue to look for additional guidelines to be added.
- Kona pre-planning:
    - Tom mentioned that he was not intending for SG16 to meet in Kona, but it looks like we'll have a quorum after all, and papers to discuss, so we will plan to meet.
    - Steve mentioned he has a paper discussing transliteration: P1439R0: Charset Transcoding, Transformation, and Transliteration.
    - Tom mentioned that Martinho and Hana have papers for us as well.
        - P1139R1: Address wording issues related to ISO 10646
        - P1433R0: Compile Time Regular Expressions
    - Peter asked why Hana's paper is targeting SG16.

    - Zach responded that the Unicode standard specifies algorithms for regular expressions.
    - Bryce suggested SG16 may want to review Hana's paper before LEWGI does.
- `char8_t` remediation:
    - Peter asked if anyone had searched for uses of `u8R`.
    - Tom replied, no, I didn't think to search for `u8` raw literals.
    - Peter mentioned having found one in the wild.
    - Victor searched Facebook and found 3 uses.
    - Tom remembered that Victor previously mentioned approximately 1000 `u8` literals in Facebook code.
    - Victor confirmed, mostly in Facebook code, mostly in tests.
    - Zach commented that he mostly uses `u8` literals in tests as well.
    - Steve stated that Chromium has a number of uses of `u8R` literals.
    - Peter added that a December 2017 snapshot of the code base he works on had 27 uses of `u8` literals, all of which were in tests. Probably has some more now, but not a lot.
    - Tom wondered why hits tend to be in tests. Perhaps library authors test things that users just don't use?
    - Zach responded that real world uses of libraries tend to use data in strings, not string literals.
    - JeanHeyd echoed Zach, data tends to come from databases or files.
    - Steve stated that most `u8` literals in the code bases he works on are in SQL queries or test code. They're still working to get off of C++03 code and have only mostly been using Unicode internally in the last 10 years.
    - Zach, stated that any changes from the `char8_t` proposal that would result in silent behavioral changes must be made ill-formed and that the remediation paper covers that. We need to be careful when discussing the remediation paper that we don't blow up concerns that aren't real. For example, with concerns that some code bases might be badly impacted.
    - Tom asked Zach how he was feeling about code like `std::string(u8"text")` now as he had previously expressed concern.
    - Zach responded that he was previously concerned about the amount of breakage, but based on anticipated impact and the fact that existing uses will now be ill-formed, no longer concerned.
- Zach asked for clarity regarding guidance SG16 gave proposal authors in San Diego regarding encoding aware interfaces.
    - Zach explained, in our response to P0881R3: A Proposal to add stacktrace library, we said that file names should be treated as just a bag of bytes. But in our response to P1275R0: Desert Sessions: Improving hostile environment interactions, we argued for an interface matching `std::filesystem::path` that exposes content in multiple encodings.
    - Corentin argued that the responses are consistent. In both cases, the content is stored as a bag of bytes, but in the latter case, interfaces are available to offer it in an encoding suitable for display.
    - JeanHeyd expressed a preference for just exposing bytes and leaving display issues up to the consumer.
    - Zach stated that the display forms encourage errors; programmers try to round-trip the names and that doesn't necessarily work.
    - Tom stated that it is good for the standard to provide encoding aware interfaces, otherwise programmers will re-invent them inconsistently.
    - Corentin observed that implimentations can provide higher quality interfaces because they can rely on platform specific behavior.
    - Tom reported having recently searched for uses of `u8string` and `generic_u8string` on Github and found only incorrect uses. For example, cases where the programmer passed the result of `generic_u8string` to `fopen`.
    - Steve said he would be in favor of deprecating the `std::filesystem::path` `u8string` and `generic_u8string` member functions for C++23.
    - Tom suggested we could deprecate them in favor of new names that indicate they are for display only. But asked if Zach thought they should exist at all.
    - Zach responded, no, they shouldn't exist as members of `std::filesystem::path`. Rather, we should have better separation of concerns and an independent interface for translating file names to

displayable strings.
    - Corentin opined that it is better for programmers to have to think about encoding issues.
- Tom verified that we'll keep with the new meeting time slot for the forseeable future and that the next meeting will be February 13th.

February 13th, 2019

Draft agenda:

- Preparation for Kona.
- Discuss P1228R1 - A proposal to add an efficient string concatenation routine to the Standard Library (Revision 1)
    - https://wg21.link/p1228r1
- Discuss P1439R0 - Charset Transcoding, Transformation, and Transliteration
    - https://wg21.link/p1439r0

Attendees:

- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jorg Brown
- Mark Zeren
- Peter Bindels
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Preparation for Kona.
    - Tom mentioned that we meet after EWGI and LEWGI have wrapped up for the week.
    - Steve observed that we can become a roadblock for proposals if we meet late in the week. That probably isn't a concern for this meeting, but could be for future meetings.
    - Peter noted that the `char8_t` remediation paper needs scheduling in LEWG.
    - Tom stated he will reach out to Titus. [Editor's note: Tom checked the LEWG schedule and P1423R0 is on the P1 priority list to be slotted in ad-hoc. Titus expects to get through all P1 priority papers]
    - Corentin asked about scheduling for P1097R2, Martinho's named character escapes proposal.
    - Tom responded that he doesn't think of it as targeting C++20 due to lack of implementation experience.
    - Zach stated it shouldn't be hard to implement.
    - Tom asked if escape sequences impact the preprocessor. Would preprocessors require updates?
    - Hubert responded that they shouldn't, though `_Pragma` is potentially impacted due to sometimes needing to reverse translation of the literal.
    - Corentin observed that the paper is already on EWG's schedule for Saturday.
- P1228R1: A proposal to add an efficient string concatenation routine to the Standard Library (Revision 1)
    - Jorg introduced the paper:
        - The proposed design has been in use at Google for 12 years and is available in Abseil as `StrCat`.

        - The design is motivated by performance and desire for a simple API. Only two overloads are proposed.
        - The design has been discussed on the LEWG mailing list.
            - http://lists.isocpp.org/lib-ext/2019/01/9692.php
            - http://lists.isocpp.org/lib-ext/2019/01/10020.php
        - The design does not have Unicode dependencies.
    - Corentin observed that there is overlap with `std::format`. Can internals be shared? Are customization points duplicated?
    - Zach stated that seems like more of a discussion for LEWG to have.
    - Corentin stated that the interface should prevent mixing types with potentially different encodings. E.g., `std::string` and `std::u8string`.
    - Victor agreed that it should not be possible to mix differently encoded strings.
    - Zach suggested that it is reasonable to only support `char` initially. Once you step outside of `char`, other locale considerations kick in.
    - Tom disagreed with locales only being a concern outside of `char` and professed agreement with the proposal that locales be a separate concern.
    - Hubert agreed with the proposal scope; avoid conversion aspects for both encoding and formatting, don't invite the complexities exhibited by stream inserters.
    - Jorg stated that having to consider locale would kill performance.
    - Tom asked if anyone wanted to argue for locale awareness and got no responses.
    - Corentin asked about dropping support for integral and float types such that only string types would be supported.
    - Hubert agreed with that direction on the basis that, with numeric types, you often want locale support.
    - Zach agreed observing that the proposed functionality seems to be conflating concatenation and formatting in a single API.
    - Hubert observed that it is useful to be able to request how much space would be needed for a numeric conversion.
    - Jorg stated that basic (non-locale aware) integer formatting is cheap (4 instructions on Intel), but not for floating point.
    - Hubert mentioned that it is also useful to be able to query a maximum buffer size for a type.
    - Jorg suggested that `std::numeric_limits` supports that.
    - Hubert disagreed; It says how many digits roundtrip, not how many might be printed.
    - Corentin observed that concatentation of differently normalized strings can change the number of perceived characters.
    - Peter asked why that is a problem.
    - Zach responded that combining NFC can result in fewer extended grapheme clusters than the two strings by themselves contained.
    - Tom expressed distaste for section III.A and treating `char` as an integral type instead of a character type.
    - Peter agreed, characters should be characters.
    - Jorg explained the direction was taken to handle `int8_t` and friends that are often defined in terms of `char`.
    - Zach suggested letting deduction work, `'0'+1` deduces as `int`.
    - Peter asked about section III.A and whether it is always customary for a minus sign to precede a negative number. In financial contexts, it sometimes follows the number.
    - Zach noted that the minus sign may appear at the end in RTL languages.
    - Tom asked for additional argumentation regarding treating `char` as an integral type vs a character type.
    - Zach suggested following the precedent set by `std:string::operator+()`; accept the kinds of arguments that it does and handle them the same way.
    - Jorg stated that, without numeric conversions, users would have to call `to_string` which is more expensive.
    - Hubert suggested the use of a proxy type when conversion is intended.

- Peter posted a link to example code he had previously written that used a rope to build a string.
  - https://github.com/dascandy/s2/blob/master/tests/string/test_simple.cpp
- Tom summarized, the dea is to use `operator+` to construct a rope that is evaluated and collapsed upon assignment to a `std::string` or other concrete type.
- Hubert noted that such approaches have difficulty with `auto`.
- Peter acknowledged that the result is rope if no conversion is specified.
- Zach asserted this is not a problem in practice and that auto can be beneficial to postpone materialization.
- Hubert stated that runs into problems with lifetime.
- Tom asked Mark about applicability to P1072.
- Mark responded, yes, P1072 explicitly mentions Abseil's `StrCat` and is instrumental to achieving desired performance.
- Jorg asked for more feedback on whether `wconcat`, `u8concat`, etc... should be provided.
- Corentin replied, yes please.
- Tom responded yes, I'd like to hear reasons not to provide them. As long as the feature remains locale independent, there are no encoding related concerns.
- Peter reiterated, for any locale stuff, let some wrapper type deal with it.
- Jorg added, or, if you want locale sensitive stuff, use `std::format`.
- Peter mentioned it would be good to update the paper with benchmarks comparing `concat` and `std::format`.
- Peter asked if `concat` needs to be concerned with `char_traits` and `allocator`?
- Zach stated that "allocators are why we can't have nice things"; ignore them.
- Tom stated that the alternative is to pass an allocator as an argument.
- Hubert suggested relying on independent type deduction for each argument, no conversions.
- Zach asked for clarification; calling `concat(1,0)` produces `"10"`?
- Jorg responded, yes.
- Peter asekd about allowing the result type to be an explicitly specified template parameter. This would allow supporting multiple string types without requiring deduction of the value type.
- Jorg expressed a preference for the option of passing an empty string as the first argument.
- Zach stated that LEWG will ask about constraints on the function. Where does SFINAE happen?
- Jorg commented that he came into this meeting only intending to support `std::string` and things easily convertible to `std::string_view`. Support for other types will require constraining the value type across all arguments.
- Tom asked for poll requests.
- Corentin suggested, do we want to support unadorned numeric conversions as arguments?
- Poll: Do we want to restrict arguments to strings, characters, and types convertible to strings.

  SF F N A SA

  3 4 1 0 1
- Jorg explained his against vote; It is really convenient to be able to simply convert integers and there is lots of usage experience in Google and Abseil.
- Corentin asked if a better `to_string` would change his vote.
- Jorg responded, pobably not as it wouldn't be as convenient.
- Peter stated that wrappers let us add features incrementally.
- Jorg mentioned that Abseil's `StrCat` provides some converters (e.g., for hex), but they are rarely used.
- Victor stated he would like to see `concat` merged with an improved `to_string`.
- JeanHeyd stated that Sol2 provides a number of adorning types but that users don't like them.

March 13th, 2019

Draft agenda:

- Post Kona review and follow up.
- RFP: Transcoding interfaces.
- RFP: Code point and EGC iterators.

Attendees:

- Bob Steagall
- Corentin Jabot
- JeanHeyd Meneide
- Martinho Fernandes
- Michael Spencer
- Peter Bindels
- Steve Downey
- Tom Honermann

Meeting summary:

- Post Kona review and follow up.
    - Tom started discussion with a proposal to initiate a draft working paper for SG16 proposals. The intent being to start drafting wording for features we'd like to get into C++23.
    - Corentin expressed concerns that a large paper could prompt calls for a TS approach and that small focused papers would be preferred.
    - Steve noted that a consolidated paper has an advantage in demonstrating how the features work together.
    - Peter observed that we will have co-dependent parts.
    - Michael expressed distaste for large papers with loosely connected features; everything shouldn't go in one paper.
    - Tom suggested brainstorming a list of features we'd like in C++23. The following were suggested:
        - `std::text` and `std::text_view`.
        - Unicode support for regular expressions.
        - Unicode code point properties.
        - Transcoding and transliteration interfaces (both compile-time and run-time).
        - A trie container.
        - A rope container.
        - Unicode algorithms.
        - Normalization.
        - A code point type.
        - A (Unicode) scalar type.
    - JeanHeyd suggested separate papers could address code point properties, a code point type, and a scalar type.
    - Martinho stated that risks getting a result that isn't useful depending on what features do/don't make it in.
    - Tom mentioned that the Design Group has stated a preference for complete and cohesive proposals.
    - JeanHeyd stated that a collection of papers can demonstrate a cohesive design.
    - Steve observed that large papers are hard to review and provided the Ranges and Networking proposals as examples.
    - JeanHeyd suggested starting with determining what we need for the 5 standard mandated encodings without support for extensions.
    - Steve noted that support for extensions may demand different interfaces for lossless vs lossy conversions and different error handling.
    - JeanHeyd countered that the ordinary and wide encodings already may be lossy, so such support is already needed.

    - JeanHeyd suggested that strong typing can be used to provide interfaces that SFINAE based on requirements; e.g., that can require non-lossy code point conversions.
    - Tom brought discussion back to the working paper idea and noted that such a paper can help drive a consensus based process.
    - Steve expressed another concern with small papers; we can end up with features being copied or re-implemented in different papers. For example, `source_location` in both the Library Fundamentals TS and in Contracts.
    - Corentin asked what the dependencies are between each of the brainstormed features.
    - Martinho stated that there is a core set that needs to be done first. Discussion identified the following:
        - Decoding and Encoding of UTF, both compile-time and run-time.
        - Unicode properties with tailoring for the private use area.
        - Normalization.
    - Corentin asked if we could postpone tailoring and dependencies on localization.
    - Martinho expressed a preference for Swift's model; default interfaces use the default locale, tailored interfaces support providing a locale.
    - Peter stated that, if tailoring is postponed, there is some risk of defining interfaces that won't work with tailoring.
    - Corentin suggested that, if we need locale support, we need to start from scratch.
    - JeanHeyd countered that, what Unicode demands from the locale is different than what is provided by `std::locale`.
    - Tom asked, for locale support, do we rely on OS settings? Or require the program to establish a locale?
    - Steve noted that, today, by default, we get the "C" locale unless `std::setlocale` is called.
    - Tom drew discussion back to the working paper idea and asked for volunteers to work on core features.
        - JeanHeyd volunteered to work on transcoding and transliteration features.
        - Corentin volunteered to work on code point properties.
        - Martinho volunteered to work on normalization.
- Discussion of D1515R0 - Code points, scalar values, UTF-8, and WTF-8
    - https://rmartinho.github.io/cxx-papers/d1515r0.html
    - Martinho presents.
        - There is a trade off. If we favor code points, then interfaces can work with WTF-8, Windows file names, and other bad data. If we favor scalar values, then we don't have to check for the presence of surrogate code points.
    - Steve asserted that we need to be able to work with ill-formed UTF text, but only at program boundaries.
    - Tom suggested there are reasons to support both code point types and scalar value types.
    - JeanHeyd agreed that both are needed, but that scalar values should be preferred.
    - Bob asked if data can be appropriately round-tripped if conversions are done at program boundaries.
    - Tom replied that a higher level protocol is needed for that.
    - Steve provided the example of CDATA in XML.
    - Martinho reported seeing use of the private use area to preserve invalid values.
    - JeanHeyd asserted that handling of invalid values should require some form of opt-in.
    - Peter responded that the opt-in is the choice of encoding and transcoding or transliteration operations.
    - Corentin asked about the motivation for both a code point type and a scalar type. Why not just rely on UB which is what contracts would provide anyway?
    - JeanHeyd asked rhetorically where the contract would be written if there wasn't a type.
    - Tom elaborated, if the scalar type is a class type, then the contract goes on constructors and assignment operators.
    - JeanHeyd asked about how to handle WTF-8 via an encoding.
    - Peter responded that you define a transcoder from WTF-8 to UTF-8.

    - Peter also suggested that, instead of using the private use area to preserve invalid values, you map them to something else, some kind of substitution character or marker. This doesn't necessarily preserve roundtripping, but allows handling.
    - Steve noted that, if transcoding/transliteration interfaces are extensible, then we don't have to solve this problem.
- https://github.com/cplusplus/draft/pull/2768
    - Tom introduced the issue; this PR to the C++ standard was marked as requiring SG16 review. The issue was created by Richard following his editorial review of the wording changes in P1139R2.
    - Tom asked that everyone take a look and offer any feedback.
- Tom announced plans for our next meeting. It will be on 3/27 and JF has volunteered to arrange for us to talk with the JavaScript team about their support for Unicode regular expressions. At this meeting, we'll discuss what we might want to ask/learn from them. If warranted, JF will then seek to arrange a future meeting.

March 27th, 2019

Draft agenda:

- Discussion with JF Bastien regarding JavaScript support for Unicode regular expressions. Based on outcome, JF can arrange for JavaScript maintainers to attend a future meeting.
- Discuss any work-in-progress from JeanHeyd, Corentin, and Martinho on transcoding, code point properties, and normalization.
- Discuss the recent LEWG mailing list emails regarding iostreams and char8_t support.

Attendees:

- Corentin Jabot
- Hana Dusíková
- Hubert Tong
- JeanHeyd Meneide
- JF Bastien
- Mark Zeren
- Michael Spencer
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Discussion with JF Bastien regarding coordinating Unicode regular expression support with ECMA TC39 and the ECMAScript standard.
    - JF introduces:
        - In contrast to what was stated in Kona, ECMAScript does specify Unicode support for regular expressions.
        - It would be beneficial to align C++ and ECMAScript support for Unicode regular expressions.
        - ECMA TC39 is currently working on adding support for Unicode sequence properties.
        - ECMA TC39 is coordinating their work with the Unicode Consortium.
        - SG16 should get involved and collaborate with ECMA TC39.
        - JF provided the following links for reference purposes:
            - https://github.com/tc39/proposal-regexp-unicode-sequence-properties

        - https://unicode.org/L2/L2018/18337-broaden-properties.pdf
        - https://www.unicode.org/L2/L2019/19056-prop-cmts.pdf
        - http://unicode.org/reports/tr18/
    - JF volunteered to connect interested SG16 participants with ECMA TC39 participants and asked for volunteers.
        - Zach expressed interest with the specifc desire for C++ to be aligned with the defacto standard (ECMAScript).
        - Hana opted in with explicit concerns regarding whether the C++ standard can defer to ECMAScript for wording specification.
        - Tom expressed interest in following along, but has no significant ECMAScript or Unicode regular expression experience to contribute; mostly intersted in staying informed for SG16 administrative purposes.
        - Corentin said, sure, why not.
    - Tom, responding to Hana's concern, stated that, by working with TC39 we can help ensure that the ECMAScript standard can be referenced normatively by the C++ standard.
    - Hana noted that, in Kona, we stated a goal of supporting UTS#18 level 1, but ECMAScript doesn't meet that.
    - Tom stated that is a good reason to work with them to find out why it isn't implemented.
    - Hana provided a link discussing why level 1 is not met:
        - https://github.com/tc39/proposal-regexp-unicode-property-escapes
    - Zach suggested that, if ECMAScript doesn't support it, we probably don't need to either. Small differences in what is supported in different languages is annoying because programmers tend to think it is ok to copy and paste between languages, but then get different behavior.
    - Zach suggested researching what level of UTS#18 support ICU provides.
    - JF noted most implementations defer to ICU and only inline simple expressions for performance.
    - Corentin observed that full UTS#18 support is madness and subsetting is necessary.
    - JF stated it would be beneficial to identify an appropriate subset of UTS#18 for both standards.
    - Hana noted that, in the link she provided, TC39 states what is required and discourages implementors from offering extensions in order to preserve portability and compatibility.
- D1628R0: Unicode character properties
    - Draft document available in the SG16 mailing list archives:
        - http://www.open-std.org/pipermail/unicode/2019-March/000266.html
    - Corentin presents:
        - Goal: Provide useful properties from UAX#44.
        - Goal: Enable querying properties for any code point for a subset of the UAX#44 properties that are deemed generally useful.
        - A reference implementation is available, but is considered early work:
            - https://github.com/cor3ntin/ext-unicode-db.
        - Hana has been using the reference implementation in CTRE to provide Unicode regular expression support in constexpr form.
        - The interface is specified as a set of predicate functions and enumerations.
        - How to handle Unicode versions is an open question. Some properties tend to be stable (e.g., general category), others are less so (e.g., script, directional, joining).
        - Corentin compared the last 5 Unicode standards for differences in property values and found little change.
        - Multiple version support is needed in order to provide a stable and portable interface while allowing for implementors to provide newer versions.
    - Michael asked about providing different versions of the algorithms as doing so could require table duplication.
    - Zach reported discovering that requirement as well. Multiple versions of the algorithms can't be provided without duplicating some tabular data. This could double the necessary storage foot print.
    - Zach observed that multiple versions of code point properties isn't useful if version specific algorithms are not provided.
    - Zach stated he didn't see a reason to expose the "age" property.

- Corentin offered two use cases for properties:
  - For use in implementing the Unicode algorithms.
  - For use by general programmers for non-algorithm purposes.
- Tom stated that it seems problematic to potentially have user code using a version other than what the standard library is using.
- Zach noted that providing multiple versions goes against our existing guidance allowing implementors to float the Unicode version.
- Steve observed that the specified interfaces are constexpr, but ICU doesn't provide data in constexpr form.
- Tom responded that the interfaces could be implemented using intrinsics that defer to ICU or a custom database.
- Corentin added that most tables are small with the name table being a notable exception. The constexpr design allows linking only what is needed.
- Tom asked, won't you still need the tables for run-time calls?
- Corentin responded that the tables could be linked in only if referenced.
- Zach stated that isn't dependable; implementors might have to link them in anyway.
- Zach listed a few specific concerns with the paper:
  - `codepoint` is marked as exposition only but can't be because it is named in specified interfaces.
  - `codepoint` needs to support `char` and `wchar_t`.
  - Interfaces taking code points should accept any integral type, encoding is not relevant here.
- Corentin explained that the `codepoint` type was introduced specifically to not allow `char` and `wchar_t` in order to avoid calls with character literals that might not be ASCII based.
- Zach stated a preference for integer values anyway.
- Tom noted that using a `codepoint` type allows using the type system to catch mistakes.
- Zach stated that type safety is illusory because `char` and `wchar_t` are code units not code points.
- Tom countered that accepting `char` and `wchar_t` is useful for character literals, but only for character literals.
- Corentin mentioned that he wants the interface to be noexcept all the way through and that wide contracts be used to avoid UB.
- Michael stated that these should be Unicode scalar values instead of code points then since any value is valid.
- Corentin agreed, the interface is defined for any integer; the predicates just return false if the value isn't a valid code point.
- Steve suggested we may want code point and scalar value types with contracts, but that probably depends on alignment with `text_view` and ranges. Maybe that is only useful at a higher level than is needed for code point properties.
- Zach predicted that LEWG will object to the `"cp_"` prefix on these interfaces.
- Zach expressed a desire for more motivation in the paper; to demonstrate a need or justification for each exposed property. Examples of why a regular programmer would care about each of these. Maintaining large interfaces or lots of properties complicates teaching.
- Corentin explained wanting to provide replacements for some broken things in the standard, like `std::isalnum`. Having these available will help programmers use Unicode properly. As an example, they are needed to implement Unicode regular expression support.
- Zach acknowledged, just want to see that motivation expressed in the paper.
- Tom agreed with adding explicit motivation like the `std::isalnum` example. Not necessarily code examples, but scenarios.
- Zach observed that some properties can be used incorrectly because they typically aren't used in isolation. Some properties should only be used via the Unicode algorithms.
- Hana stated a preference for exposing the Unicode standard as it is specified. Considerable work and expertise has gone into it. It is a standard that people can learn.
- Zach disagreed on a philosophical basis; want to keep things simple.
- Steve observed that some of this is ergonomics of naming. Programmers should reach for a function first, then raw properties only if necessary.

    - Zach cautioned about exposing an expert-only interface.
    - Corentin mentioned, by defering to Unicode, we avoid making mistakes. Properties that are only used for derived properties are already excluded.
    - Tom stated that we can always expose additional properties later as we identify use cases.
    - Michael stated that some properties are implementation detail within the Unicode standard; they exist for the algorithms to refer to them. We should focus first on high level interfaces and those probably won't be defined in terms of low level property interfaces.
    - Zach stated that properties that are only needed to implement an algorithm need not be exposed individually.
    - Tom asked about tailoring and properties for the private use area (PUA).
    - Corentin replied that tailoring should be provided by a separate interface. The PUA shouldn't be used in open interchange.
    - Tom agreed, but stated that, within an application, a programmer might want all libraries to see the same customized properties for the PUA.
    - Steve noticed that the Unicode version numbers in the `version` enumerator values jump from `0x09` to `0x10` rather than to `0x0A`.
    - Corentin replied, oops.
    - Corentin added, if we don't support multiple Unicode versions, there is a question of enumeration value stability across implementations and differing Unicode versions.
    - Tom suggested that we'd like to see a revision of the paper and asked for objections.
    - No objections were raised.
- D1629R0: Standard text encoding
    - JeanHeyd screen shared an early draft that has not yet been published, so no link is available.
    - JeanHeyd presents:
        - The proposed design follows review of a number of prior papers and projects:
            - P0244 - Text_view: A C++ concepts and range based character encoding and code point enumeration library
            - text_view
            - libogonek
            - Boost.Text
        - Enabling optimizations is a goal.
        - Want a range based approach. Want to enable lazy encoding/decoding.
        - Wrapping iterators can be large, iterator/sentinel pairs are helpful to reduce iterator sizes.
        - Can't depend on locale or `codecvt` because of performance costs; `wstring_convert` exhibited these costs in Sol2.
        - Exposing state enables chunked streaming.
        - An empty state can indicate a self-synchronizing encoding.
        - State could be potentially omitted in interfaces for stateless encodings.
        - The default error handler will substitute replacement characters.
        - Error handling can be elided by specifying an assume_valid handler.
        - Sized output ranges can be used for memory safety.

April 10th, 2019

Draft agenda:

- Continue discussion of JeanHeyd's D1629R0: Standard text encoding
- Discuss any further work-in-progress from Corentin and Martinho on code point properties, and normalization.
- Discuss execution encoding, current locale dependency, and the feasibility of mandating UTF-8.

Attendees:

- Corentin Jabot
- Hana Dusíková
- JeanHeyd Meneide
- Mark Zeren
- Steve Downey
- Tom Honermann

Meeting summary:

- Continue discussion of JeanHeyd's D1629R0: Standard text encoding
  - JeanHeyd walked us through the code he has been developing to prototype interfaces to be proposed.
    - https://github.com/ThePhD/phd/tree/master/include/phd/text
    - Encoding types have `encode()` and `decode()` member functions that accept an input range, an output range, a state, and an error handler and return an `encoding_result` type that forwards the possibly mutated input range, output range, and state. These functions operate on a single code point at a time.
    - `text_transcode` and `text_transcode_into` interfaces are provided for conversion of multiple code points at a time. Generic implementations are provided, but the design is intended to support optimizing for contiguous ranges or characteristics of specific encodings.
  - Tom asked if input and output iterators/ranges are supported or if forward iterators/ranges are required.
  - JeanHeyd replied that input and output iterators are supported, but an error handler won't be able to observe the code units that provoked invocation of the error handler.
  - Tom suggested that a caching iterator can be used to solve that problem. This is the approach used by text_view.
  - Steve added that the state type can also be used to cache such code units.
  - JeanHeyd explained more about the error handling. `encoding_result` can store an error status and any additional useful information. The implementation uses the facilities provided by the `<system_error>` header.
  - Tom asked for clarification; ranges are always moved into and back out of error handlers?
  - JeanHeyd answered, yes, via `encoding_result`.
  - Tom asked about the possibility to encode only a state change without encoding a character.
  - Steve asked for clarification; as in for a ISO-2022 style shift sequence?
  - Tom confirmed.
  - JeanHeyd replied that the state type can be used for whatever purposes.
  - Tom expressed skepticism about that working from an interface perspective since both input and state are provided. Sometimes, you just want to encode a state transition.
  - Steve noted that state is strongly tied to encoding and asked if state needs to be exposed in the interface. The ability to resume a conversion is still necessary, but wouldn't have to be handled via state.
  - JeanHeyd stated that having the state be separate is useful for flexibility in resumption.
  - Steve added that state often becomes a house keeping burden. Users generally don't know how to work with it and do things like passing an initial default constructed state when a resumption state is needed.
  - Tom asked how much JeanHeyd had reviewed text_view as some of what is being discussed appears to be reinventing solutions implemented there.
  - JeanHeyd responded that the interfaces were influenced by reviews of Boost.text, text_view, and Ogonek.
  - Corentin asked if the transcoding interfaces can provide lazy ranges.
  - JeanHeyd responded no, not yet.

- Steve stated that lazy ranges don't necessarily play well with optimized transcoding operations.
- JeanHeyd stated that the interface is intended to allow optimization. Implementors can use `if constexpr` internally.
- Tom expressed concern about reliance on `if constexpr` within an encoding agnostic generic function and suggested specialization as a more extensible solution.
- JeanHeyd explained that overloading can be used to provide more optimized implementations without lots of specializations.
- Tom observed that the trade off is a bunch of overloads vs a bunch of specializations.
- JeanHeyd acknowledged the trade off, but noted that often fewer overloads are required because conversions can be relied on.
- Tom suggested that, perhaps, there should be a `std::transcode` customization point.
- JeanHeyd acknowledged and said he is still playing around with such ideas and wants to enable users to provide custom overloads.
- Tom expressed hope that users won't be writing these often at all; that such interfaces should mostly be written by library providers.
- Steve agreed and added, or small infrastructure teams.
- Steve asked about type erasure and support for dynamic encodings.
- JeanHeyd expressed uncertainty about the need to provide that within the standard and illustrated how a custom encoding that handles dynamic encodings could be written.
- Steve added that POSIX provides iconv which allows requesting a codec for a named encoding and such functionality should be provided.
- Tom suggested it might suffice to be able to write an `iconv_encoding` type that wraps iconv.
- Tom asked how transcoding between encodings with different associated character sets are handled.
- JeanHeyd responded that no support is present yet, but that attempting to transcode between such encodings would fail compilation if the code point types were not compatible.
- Tom observed that failing compilation requires a strong type, not `char32_t`, to enforce safety via the type system.
- Corentin asserted that `basic_text_view` should not have a template parameter for normalization since normalization is not relevant for all encodings.
- Tom agreed that, if present at all, normalization should be incorporated into the encoding type.

May 15th, 2019

Draft agenda:

- Continue discussion of UTF-8 as execution encoding. Our focus last time was on impediments to use of UTF-8 as execution encoding. Focus this time will be on anticipated benefits of mandating UTF-8 and impact to existing ecosystems.

Attendees:

- Cameron Gunnin
- Henri Sivonen
- Hubert Tong
- JeanHeyd Meneide
- JF Bastien
- Michael Spencer
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- Discussion of potential benefits/costs of mandating UTF-8 as execution encoding:
  - Tom introduced the topic with a brief summary from the last meeting.
  - Zach stated he was all for moving to UTF-8.
  - Tom asked how code would be written differently compared to today.
  - Zach presented some problematic examples he has encountered in the past. The first was surprises with UTF-8 encoded literals in source code not retaining UTF-8 encoding at run-time. The second was difficulties writing text and having it display in terminals as expected. Today's compilers don't have options to state that the output of a program will have any particular encoding. Writing non-ASCII output is expert territory. We can't write portable code that dumps text and expect it to just work.
  - Tom noted some subtleties with those examples; there are actually four encodings involved. Source encoding, presumed execution encoding, run-time execution encoding, and terminal/console encoding. This raises the question of what is meant by mandating UTF-8. Mandating it for all of these encodings?
  - Steve stated that these issues affect all platforms. Use of characters outside the basic source character set doesn't work unless `std::setlocale()` is called to set a locale other than `"C"`. Changing the default locale to `"C.UTF-8"` would be an improvement as it would suffice to make the multibyte conversion functions work as expected without changing behavior for character classification functions. This matches what Python is doing as described in PEP-538 and PEP-540. This still allows the locale to be run-time selectable, but provides a better default for character encoding.
  - Tom commented that this would preserve existing behavior since, today, unless `std::setlocale()` is called to change the current locale, characters outside the basic source character set elicit undefined behavior for the multibyte conversion functions.
  - Zach asked how this would be proposed. By defining a "C.UTF-8" locale? Or by specifying that the default "C" locale operate as if `LC_CTYPE` were set to UTF-8?
  - Steve responded that the implementation behave as though an implicit call to `std::setlocale(LC_CTYPE, "C.UTF-8")` occurred during process startup.
  - Henri observed that the behavior of `nl_langinfo` would be affected by doing so; `nl_langinfo(CODESET)` would now return a string reflecing UTF-8 rather than the encoding used to implement the "C" locale.
  - Zach noted that such a change needs discussion with WG14 and POSIX members. It would not be good if behavior differed based on whether C or C++ headers were included.
  - JeanHeyd indicated that he is already working on such a discussion and plans to submit a paper to WG14 proposing that "C.UTF-8" be made a standardized locale.
  - Steve noted that POSIX exposes more encoding aware facilities than C does; more character classification functions for example.
  - Zach asserted that, if we made UTF-8 the default, life would be easier for everyone.
  - Tom summarized discussion so far; we've been discussing changing the default locale, but not mandating UTF-8.
  - Hubert noted that mandating UTF-8 will affect presumed encoding.
  - Henri suggested that mandating a particular encoding might solicit reluctance from implementors. If implementors don't go along, then the standard doesn't match reality.
  - JeanHeyd agreed with the concern and noted that changing the default leaves open an escape hatch for preserving existing behavior. While mandating a particular encoding would make some things easier, it would also leave some platforms and/or implementations behind.
  - Tom asked Hubert for his perceptions regarding changing the default; how would the platforms he supports be impacted?
  - Hubert replied that, on z/OS, it would be kind of odd due to the possibility of multiple processes sharing the same language environment.
  - Tom asked what happens today if a single process changes its locale.

- Hubert responded with some uncertainty. Switching between EBCDIC code pages is much less impactful than switching from an EBCDIC code page to something ASCII based would be.
- Hubert returned to questions of implementation; how would the implicit call to `std::setlocale()` be implemented? This isn't a typical language level thing.
- Tom responded that such a question is what he would have posed to Hubert.
- Hubert elaborated on potential complexities. How would this work when separately compiled components potentially compiled for different standard versions are linked together? Is this a linker option?
- Tom stated that is outside the scope of the standard.
- Hubert agreed, but noted that concerns like this don't come up very often.
- Zach reiterated the intent; that the C++ startup code perform as if an implicit call to `std::setlocale()` took place.
- Hubert acknowledged the intent, but noted the potential unintended effects that Henri eluded to earlier. For example, on AIX, Hubert tried a program that displays the current locale encoding. When invoked with `LANG=C`, it indicated ISO-8859-1. An implicit call to `std::setlocale()` would change this behavior.
- Tom said he would like to see a concrete example of that behavior.
- *[Editor's note: Tom later experimented on a Linux system and observed the same behavior:*

```
$ cat t.cpp
#include <langinfo.h>
#include <locale>
#include <cstdio>

int main() {
  std::printf("%s\n", nl_langinfo(CODESET));
  std::setlocale(LC_CTYPE, "");
  std::printf("%s\n", nl_langinfo(CODESET));
}

$ g++ t.cpp -o t

$ LANG=C.UTF-8 ./t
ANSI_X3.4-1968
UTF-8
```

  ]
- Zach suggested that the multibyte conversion functions don't really work now, so changes can be made.
- Hubert disagreed and stated they work exactly as intended.
- Steve stated that functions like `std::wctomb` will silently drop or replace characters for the "C" locale.
- Tom asked Zach if this is an example of what he meant when he said they were broken.
- Zach replied, yes.
- Henri noted that silently dropping/replacing characters can be a security issue. By specifying behavior, we may reduce security problems.
- Hubert agreed that, if you don't do proper error checking, that can lead to problems.
- Tom asked if these functions have appropriate error handling interfaces.
- JeanHeyd stated that they do, they return -1 on error.
- Tom clarified, yes, they return -1 if an ill-formed code unit sequence is encountered, but do they also return in error if a character lacks representation in the current locale and a replacement character is substituted? Tom thought at least some implementations substituted replacement characters without errors.
- Tom summarized the ramifications of an implicit `std::setlocale()` call; doing so is a breaking change given Henri and Hubert's observations about querying the current locale encoding.
- Hubert agreed, noting that non-exotic platforms are impacted.
- Steve suggested that the observeable impact should be limited to querying locale properties.

- Hubert stated that may be true, but that we haven't discussed presumed execution ncoding yet. The standard only discusses one execution encoding, but there are two.
- Tom acknowledged and added that we know this to be problematic as it imposes a lowest common denominator approach for encoding literals. If the presumed execution encoding were UTF-8, that would clearly be problematic on z/OS, but even on ASCII based platforms, if the locale can be changed at run-time, that limits use of UTF-8 in literals.
- Hubert commented that the fallout for z/OS for changing presumed execution encoding may not so bad because extensions are already available to specify execution encoding and to opt into an ASCII based encoding. Additional support might be needed for locale encodings.
- Tom asked to clarify what was meant by additional support.
- Hubert responded that he was thinking about message catalogs used with translation. Generally, messages can be written in a common subset of characters available in all locales. If we mandated an encoding that doesn't share a common subset (e.g., UTF-8 on a predominantly EBCDIC based platform), then strings would have to be maintained in resource files outside of translation units, or written in escape sequences.
- Tom asked if Unicode escape sequences are used much on z/OS.
- Hubert responded that, in newer code, yes, but generally only in Unicode literals, not in ordinary or wide literals.
- Tom returned to the question of benefits of mandating a single execution encoding. If we did so, then we could make text processing decisions at compile-time.
- JeanHeyd suggested that we can do that anyway.
- Tom responded that, no, evaluation of functions that are locale dependent must be deferred until run-time since the encoding is not known.
- Tom asked about the feasibility of changing the default locale given existing ecosystems. We would have to work with POSIX, WG14, implementors, other languages?
- Zach reiterated that he wants to see the portability problems writing to terminals be fixed.
- Tom asked, but isn't that a problem not specific to C++? All languages face this issue.
- Zach noted, for all processes, data comes out as bytes, but terminals can't handle them.
- JeanHeyd noted that codecvt facets may interfere, but the primary difficulty programmers face on Windows is that the default terminal settings and execution encoding are locale dependent.
- Zach observed that, if everything was UTF-8, then the terminal could just do the right thing. But we can't mandate behavior for all languages.
- Henri asked if Microsoft's new terminal might help.
- Tom stated that it should, but will require some form of opt-in. Historically, Microsoft's default console encoding has been constrained by backward compatibility. The encoding of the console differs from the locale encoding in order to support those old applications that depend on line drawing character sets. Unclear how new applications will opt-in to UTF-8 behavior.
- Zach suggested that just having a way to determine if the current locale supported UTF-8 would be useful.
- Henri asserted that the experience with Python demonstrated that such approaches don't work in practice. Asking the C environment what the execution encoding is rather than just assuming it may actually cause more problems. What would you do if the current encoding wasn't compatible?
- Zach replied that, if the encoding is known, then the program can convert.
- Tom stated that is how the multibyte conversion functions are already specified to work; if they are used, then the output will match locale encoding.
- JeanHeyd expressed having unsatisfactory experience with the multibyte conversion functions. For example, Microsoft's implementation of `mbrtoc32` unconditionally converts between UTF-8 and UTF-32 in violation of the standard. The conversion functions are also unable to detect some kinds of mojibake; for many single byte encodings, all possible code unit sequences are valid.
- Hubert concurred that the multibyte conversion functions are not a good alternative to ICU. Discussions about `source_location()` are touching on a bigger problem; C++ is just one language residing in a large ecosystem in which implementors are beholden to the platforms they support.
- Henri asked about wide characters being compatible across locales. What encoding does z/OS use for them?

    - Hubert deferred to documentation and noted that, on AIX, the wide character set does vary for some Chinese locales.
    - Tom stated that the wide character set on z/OS is a wide form of EBCDIC with support for ISO-2022 escape sequences.
    - Hubert followed up regarding the wide character sets used on AIX. Depending on the size of `wchar_t`, either UTF-16 or UTF-32 is used except for the Taiwanese locale which uses Big-5.
    - Tom asked JF what benefits Apple might see from changing the default locale encoding to UTF-8.
    - JF responded that there would be some simplicity.
    - Tom pondered if the biggest benefit to Apple would come from dropping locale dependent encoding completely.
    - Hubert asserted that isn't feasible; IBM customers are known to create their own custom locales.
    - Steve noted that one way in which applications are meeting the requirement for GB18030 by the PRC is via locales.
    - Henri expressed a belief that the PRC doesn't actually require GB18030 support; rather that the mandatory character subset from it be supported.
    - Steve stated that he was required to add support for GB18030 and did so via conversions at program boundaries.
    - JeanHeyd suggested that no one is really looking to deprecate locale support. We just want to make text processing easier and better locale facilities would help.
    - Henri stated that he would like to deprecate broken character classification functions like `isalpha()` since they can't properly handle Unicode inputs.
    - JeanHeyd suggested that, if we had new and improved locale primitives, then we could deprecate the old ones.
- Discussion then moved on to an issue that JF raised on the mailing list concerning Unicode characters in identifiers.
    - http://www.open-std.org/pipermail/unicode/2019-May/000367.html
    - https://github.com/sg16-unicode/sg16/issues/48
    - Tom briefly introduced the topic. The C++ standard specifies a subset of Unicode characters that may be used in identifiers. That specification is via a series of code point ranges in [lex.name]p1, but the standard doesn't offer a rationale for the specified ranges. It seems likely that the ranges aren't being maintained as Unicode evolves.
    - JF stated that the current ranges lack justification, but that isn't much of a problem in practice. Clang accepts identifiers using the specified code points, but gcc does not, so such identifiers are not portable in practice. Perhaps the standard should just adopt and defer to UAX#31.
    - Henri asked what the motivation is to address this if it isn't a problem in practice.
    - JF responded that it is a problem in principle. It is unclear why gcc allows the code points in identifiers if specified via `\u` escapes, but not when the actual characters are present in the source encoding. If the standard was more precise, perhaps implementations would be more consistent. Also, such characters should be allowed in module names and we should use the same identifier scheme there.
    - Henri asked how much of the concern is separating identifiers from operators.
    - JF responded little since there are no plans to, for example, adopt Unicode math symbols as operators.
    - Hubert provided a little history. The existing code point ranges were provided by Clark Nelson in WG14's N1518 and were based on the version of UAX#31 current at that time.
    - Hubert stated that it would be annoying to have the standard defer to a Unicode TR as doing so makes it that much more difficult to determine what the actual rules are for a given standard.
    - Tom suggested that deferring to a Unicode TR would at least have the benefit of the ranges matching whatever version of the Unicode standard the implementation adheres to. This would presumably help maintain the ranges as characters are added over time.

May 22nd, 2019

Draft agenda:

- Axel Andrejs from Microsoft will present and discuss Microsoft's ongoing efforts to improve UTF-8 support in Windows.

Attendees:

- Axel Andrejs
- Henri Sivonen
- Hubert Tong
- JeanHeyd Meneide
- Mark Zeren
- Steve Downey
- Tom Honermann

Meeting summary:

- Microsoft's on-going efforts to improve UTF-8 support in Windows.
    - Axel lead with some Windows history and current efforts:
        - Windows was initially developed using locale dependent code pages, but switched to UTF-16 long ago.
        - Today, the industry is moving to UTF-8, especially on the web.
        - So, what to do about UTF-8? Initial efforts were to improve resource managers, but Windows remains UTF-16 based.
        - Now, what about the rest of the OS? Windows 10 added support for UTF-8 as an (algorithmic) active code page (ACP).
        - All of Windows' "ANSI" interfaces use `MultiByteToWideChar()` to convert `char` based input to UTF-16 using the active code page. This suffices to make UTF-8 mostly magically work, though Microsoft platforms remain UTF-16 based.
        - Within the industry, most Windows developers don't test support for many code pages; usually just a few for critical markets. This leaves a large testing gap.
        - UTF-8 poses some problems as a code page. Testing UTF-8 support has revealed cases of code that fails due to a number of issues:
            - It is variable length and may require more than two code units per code point. Code that (incorrectly) assumes wide strings are always larger (in terms of bytes) than the corresponding narrow encoding can cause buffer overflows.
            - The native C and C++ character type is `char` which is good in terms of flexibility, but bad in terms of type and encoding safety. `char` can be used for UTF-8, but failure to perform conversions where needed leads to mojibake.
            - Some code just plain fails with any non-ASCII characters.
            - Many programs assume an encoding (ASCII or Windows-1252) and fail with UTF-8.
        - Because of known cases of existing code failing when the active code page is changed to UTF-8, Microsoft has no plans to ever change an existing machine's active code page. There is too much potential for breakage.
        - Current efforts include improving UTF-8 support for individual Windows components, resource managers, resource loaders, etc... UTF-8 strings tend to be approximately 1/3 the size of wide strings on average. Focus on where that savings is beneficial.
        - UTF-8 as active code page is still a beta option because there are major applications that don't work correctly when it is enabled. Most applications work ok in the US, but have problems elsewhere.
        - Outside Windows desktop where compatibility with legacy applications is less important, Windows platforms are moving more towards UTF-8. These include Xbox, Hololens, smart

devices, etc...
    - Bifurcation will continue. We may evangelize UTF-8 in some markets, but are unlikely to ever be able to move the entire Windows ecosystem to UTF-8.
    - The latest Windows 10 release allows executables to opt-in to UTF-8 as active code page via a fusion manifest.
    - May add support for executables to opt-out of UTF-8 support for compatibility, either via a manifest setting or application compatibility shim.
    - Windows subsystems will continue to migrate to UTF-8.
    - Would like to discontinue the ANSI/Wide interface split. Likely to introduce more interfaces that are UTF-8/Wide or just UTF-8.
    - There are no plans for a native Windows UTF-8 kernel.
    - We'll continue to reach out to major applicaions that are found not to work correctly when the active code page is set to UTF-8.
    - Within Microsoft, a number of developers have UTF-8 enabled as the active code page on their workstations.
    - We will continue to do more ecosystem outreach as our confidence in UTF-8 as active code page increases.
    - But, Windows will always need to retain compatibility.
- Henri asked if there are plans to augment existing ANSI/Wide interfaces with U8 variants that only work with UTF-8.
- Axel responded that they would like to do so where it makes sense. Decisions to do so are up to component owners. Feedback and requests for specific interfaces are appreciated. There are no plans for mass conversion.
- Henri asked what would happen if an executable that was marked to run with UTF-8 was run on an older platform.
- Axel replied uncertainly that the fusion manifest entries would probably just be ignored. This is what happens with Universal Windows Programs (UWP) support. Unsure if there is a way to mark the executable to fail in such cases, but the executable could be marked to require a particular level of OS support.
- Mark asked how an executable that opts-in to UTF-8 as ACP would interoperate at the command line. Is any transcoding performed for stdin/stdout?
- Axel responded that, at present, all the opt-in does is override the ACP, so no, no transcoding of the command line or standard streams is performed.
- Mark observed that this can then lead to failures.
- Axel affirmed adding that thought has been put into implicit transcoding, but it is a hard problem.
- Tom agreed noting that file names pose a significant problem since they may not be representable in a particular encoding.
- Axel acknowledged, but stated that most file names are UTF-16.
- Henri asked about the new terminal coming to Windows 10. How will it know how to interpret the output of a particular program?
- Axel stated that there are active discussions about this and decisions are not yet settled, but people are working on it.
- Tom asked about recommendations for current developers. How do we move into this new world?
- Axel responded that `char*` is kind of nice for it's genericity but isn't safe. Stronger types add safety, but increase the interface surface area. `char8_t` is a big topic. ICU supports both `char16_t` and `wchar_t`, but adds surface area. As a global model, that doesn't work too well. If targeting Windows only, best to stick to wide strings. For cross platform, we're all on this journey. Would like more feedback. There are always workarounds because developers can do conversions themselves. If there is demand, we'll add additional interfaces if the value is there. Would like more support for command line handling. Really want the industry to move to UTF-8 as ACP. Library writers have to worry about all code pages anyway, including now UTF-8.
- Mark asked how new types like `char8_t` fit in.
- Axel expressed similar curiosity.

- Tom provided some thoughts on `char8_t`. Unsure what kind of adoption will occur. Expecting to see uses in niches or as an internal encoding type. Type safety can be used to guard components that are UTF-8 only from components that are locale dependent.
- Henri asked if type based alias analysis is planned for the Microsoft compiler.
- Axel responded that he was unsure of the compiler team's plans. Windows interfaces have pretty basic types, so there is a lot of targeting lowest common denominator. Always looking to take advantage of new features.
- Mark offered the idea of a compiler option that would allow use of `char8_t`, but would mangle it the same as `char` for compatibility with prior compiler versions. This would require errors for ambiguous overloads, but might still be useful.
- Tom expressed interest noting that Microsoft already does something similar to duplicate symbols for `char16_t` and `wchar_t`.
- Axel confirmed noting that they sometimes put code in headers and compile it twice (e.g., for ANSI and UNICODE expansions of `TCHAR`).
- Henri asked why `char16_t` was not made the same type as `wchar_t` on Windows.
- Hubert responded that C++ requires different types for overloading purposes. Also because the encoding can differ.
- Henri pondered whether, in retrospect, it would have been better if char16_t was specified as the same type as `wchar_t` on Windows and `char32_t` the same type as `wchar_t` on POSIX systems.
- Steve stated that anyone attempting to write portable code is unhappy with `wchar_t`; it just isn't portable.
- Mark added that the type separation is useful. For `char8_t`, the non-aliasing properties are a good motivator for a separate type.
- Steve concurred noting that injecting Unicode into the type system will be useful. Additionally, `std::byte` will help us move further away from `char` for everything.
- Axel mentioned that, on Windows, there are few APIs that take `char*` and that expect an encoding other than the ACP. The only indication is in documentation; the type system can't help enforce encoding expectations today.
- Mark asked what code page is used for Windows Subsystem for Linux (WSL).
- Axel responded that he would have to check, but once code reaches the kernel, everything is UTF-16.
- Tom observed that different encoding expectations in Windows vs the WSL makes piping data problematic.
- Mark surmised that the WSL uses UTF-8 like most Linux distributions.
- Axel added that the International Platform team didn't do anything special for WSL.
- Mark speculated that, if we decided to be very aggressive, we could require C++ code on Windows to run with the UTF-8 manifest option.
- Axel confirmed, but noted that requires new OS versions.
- Tom asked Axel if his team had reached out to other language maintainers like for Python, Ruby, and Go.
- Axel responded yes for .NET languages obviously, but not for other languages. Need to reach critical mass internally first, and then will expand outreach to other languages.
- Henri asked about enabling app development with UTF-8 on older OS versions. Are there any plans for the standard library to provide UTF-8 interfaces that convert to internal UTF-16 ones?
- Axel responded that work is progressing to improve support for UTF-8 in the CRT, but not sure of the time line.
- Mark asked about any work on interfaces that operate at the grapheme cluster level.
- Axel responded no, at present, they are more focused on basic APIs like `CreateProcess`.
- Tom asked how SG16 can help with the effort to improve UTF-8 support.
- Axel explained that the big challenge is how to handle the lowest common denominator. New language features are used internally, but public APIs are very old school and limited to basic types.
- Tom clarified, so keeping interfaces indpeendent of fancy new language features is helpful?
- Axel responded yes, but always interested in new types that make sense within the Windows type system.
- Tom summarized all of the different encodings that the C++ standard has to interact with; source encoding, internal compiler encoding, presumed (compile-time) execution and wide execution encodings, (run-time) execution and wide execution encodings, UTF-8, UTF-16, and UTF-32. How do all of these encodings affect you?

- Axel responded that they sometimes have to guess about encodings; may rely on BOMs or recognition of UTF-8.
- Tom asked if Microsoft had ever considered allowing filesystems to support tagging files as having a particular encoding. Some other OSs like z/OS have such support.
- Axel responded no, not aware of any such efforts.
- Mark asked if Windows makes use of the Unicode Private Use Area (PUA).
- Axel responded yes, because, given the size of the development team at Microsoft, the answer to "do we use ..." is always yes somewhere.
- Henri commented that Microsoft's eudcedit.exe editor generates a magic font for the PUA.
- Tom asked if Axel had any thoughts about changing the default "C" locale to be UTF-8.
- Axel responded that it might work out ok. Old CRTs still get used though. But this would make it easy for programmers to start using UTF-8.
- Mark noted that Microsoft maintains backward compatibility, but that there may be some desire or intent for an ABI break at some point. Perhaps that would be the right time to change the default locale encoding.
- Henri asked if layering new versions of C++ on top of older C and C++ run-times is supported or whether new C++ language standards require the latest C and C++ run-time libraries. Would it be possible to have the compiler set the per-binary UTF-8 flag depending on target language level?
- Axel responded that the UTF-8 flag affects the process, so can't depend on DLL options or settings.
- Mark brought up a new issue; at some point, we will require various Unicode data sets. Windows now distributes a version of ICU. Concerns about the size of the chrono library were raised when it was added to the standard and it is much smaller than the Unicode data set. We'll likely require at least data for normalization and collation.
- Axel provided some additional background. Windows has NLS interfaces. ICU was added to Windows 10 two years ago, but application developers still want to target Windows 7 where it isn't available. Windows 10 now supports about 3-4 times more locales than Windows 7 due to the inclusion of the CLDR. Carrying ICU with an application is becoming a significant servicing issue. Time zone data bases were integrated into Windows to address similar servicing issues, but have to be updated often and quickly for geopolitical reasons. CLDR data is unlikely to be updated as frequently.
- Henri asked for more details. How is ICU updated in Windows 10? Will older Windows 10 releases get updates?
- Axel responded that the latest available ICU is distributed in each 6 month release cycle, but that locale data and Unicode versions are not otherwise patched. Exceptions could be made, but would require significant motivation like the geopolitical reasons that motivate time zone updates.
- Henri surmised that older Windows 10 releases won't get support for new Unicode characters, data tables, etc...
- Axel confirmed.
- Hubert stated that he had heard that the ICU distributed in Windows 10 does not exactly match any official ICU release.
- Axel confirmed adding that patches are made for geopolticial reasons.
- JeanHeyd stated that the road to UTF-8 support is going to be a long one. He plans to take papers to the C and POSIX committees proposing to change the default locale to UTF-8 in order to facilitate a similar change for C++. The goal is to allow applications to at least be able to communicate without mangling text. It sounds like Microsoft is heading in a good direction, but the C committee may be reluctant to make such a change.
- Axel agreed that this will be a long journey and encouraged everyone to play with the new UTF-8 functionality, report problems, and poke application providers to improve support. The problems are not massive, but cost/benefit analysis must line up as always. Application providers will be required to support UTF-8 as ACP for some Microsoft platforms like the Xbox and Hololens.
- JeanHeyd commented that our biggest concern has been how to migrate to a UTF-8 world. At least it sounds like there is a path to follow.